ESQUECI A SENHA e REDEFINIR SENHA

Etapa 1 — Entender o fluxo completo

1. Cliente informa o e-mail em Esqueci minha senha.

2. POST /auth/esqueci-senha solicita o código e, se houver sucesso, abre o modal.

3. Ao continuar/fechar o modal, o e-mail é passado para /redefinir-senha por parâmetro.

4. Cliente informa os quatro dígitos, nova senha e confirmação.

5. Ao tocar em Salvar senha, POST /auth/validar-codigo retorna reset_token.

6. Na mesma função, POST /auth/redefinir-senha utiliza reset_token e as duas senhas.

7. Somente após resposta positiva, o app retorna ao login.

Etapa 2 — Conferir o arquivo e a configuração da API

Abra src/app/esqueci-senha.tsx. Confira os imports de API e router.

```
import { useState } from "react";
import { router } from "expo-router";
import { API } from "@/config/api";
```

Etapa 3 — Manter os estados e o campo de e-mail

```
  const [modalEnviarLink, setModalEnviarLink] = useState(false);

  const [email, setEmail] = useState("");
  const [carregando, setCarregando] = useState(false);
  const [mensagem, setMensagem] = useState("");
```

No TextInput de e-mail, value={email} mostra o estado atual; onChangeText={setEmail} atualiza o estado ao digitar:

```
                  <TextInput
                    placeholder="E-mail"
                    placeholderTextColor="#888888"
                    keyboardType="email-address"
                    autoCapitalize="none"
                    style={esqueciSenhaStyle.textInput}
                    value={email}
                    onChangeText={setEmail}                    />
```

Etapa 4 — Solicitar o código na primeira tela

Dentro de EsqueciSenhaScreen(), mantenha a função abaixo. Ela valida campo vazio, envia o POST, trata respostas e abre o modal somente se a API responder com sucesso.

```
  async function solicitarCodigo() {
    if (!email.trim()) {
      setMensagem("Informe seu e-mail.");
      return;
    }
```

```
    setCarregando(true);
    setMensagem("");

    try {
      const resposta = await fetch(`${API}/auth/esqueci-senha`, {
        method: "POST",
        headers: {
          "Content-Type": "application/json",
          Accept: "application/json",
        },
        body: JSON.stringify({ email: email.trim() }),
      });

      const json = await resposta.json();

      if (!resposta.ok || !json.success) {
        setMensagem(json.message || "Não foi possível solicitar o código.");
        return;
      }

      setMensagem(
        json.message ||
          "Se o e-mail estiver cadastrado, enviaremos um código de verificação.",
      );
      setModalEnviarLink(true);
    } catch (erro) {
      console.log("Erro ao solicitar código:", erro);
      setMensagem("Não foi possível conectar ao servidor.");
    } finally {
      setCarregando(false);
    }
  }
```

Etapa 5 — Conectar botão, mensagem e modal

O botão Enviar código chama solicitarCodigo, é desabilitado durante o envio e mostra Enviando... enquanto a requisição acontece. A mensagem fica dentro do formulário:

```
                {mensagem ? (
                  <Text style={esqueciSenhaStyle.erroSolicitacao}>
                    {mensagem}
                  </Text>
                ) : null}
```

```
                <Pressable
                  style={({ pressed }) => [
                    esqueciSenhaStyle.btnEnviar,
                    pressed && esqueciSenhaStyle.btnEnviarPressed,
                  ]}
                  onPress={solicitarCodigo}
                  disabled={carregando}
                >
                  <Text style={esqueciSenhaStyle.textoEnviar}>
                    {carregando ? "Enviando..." : "Enviar código"}
                  </Text>
                </Pressable>
```

Mantenha o modal existente, mas o texto interno deve mencionar código, não link. O evento onClose deve levar o e-mail para a segunda tela:

```
          <EnviarLinkModal
            visible={modalEnviarLink}
            onClose={() => {
              router.push({
                pathname: "/redefinir-senha",
                params: {
                  email: email.trim(),
                },
              });
              setModalEnviarLink(false);
            }}
          />
```

Etapa 6 — Preparar a segunda tela para receber o e-mail

Abra src/app/redefinir-senha.tsx. Confira os imports, estados e referência dos campos de código:

```
import { useState, useRef } from "react";
import { router, useLocalSearchParams } from "expo-router";
```

```
  const [verSenha, setVerSenha] = useState(false);
  const [verConfirmarSenha, setVerConfirmarSenha] = useState(false);

  const { email } = useLocalSearchParams<{ email?: string }>();
  const [codigo, setCodigo] = useState(["", "", "", ""]);
  const [novaSenha, setNovaSenha] = useState("");
  const [confirmarSenha, setConfirmarSenha] = useState("");
  const [mensagem, setMensagem] = useState("");
  const [carregando, setCarregando] = useState(false);
  const camposCodigo = useRef<Array<TextInput | null>>([]);
```

useLocalSearchParams recupera o parâmetro email da rota. Ele não deve ser substituído por uma variável inexistente chamada emailRecuperacao.

Etapa 7 — Implementar os quatro dígitos no mesmo formulário

Os quatro campos já são renderizados a partir do array codigo. As funções abaixo aceitam somente números, avançam o foco e permitem voltar com Backspace:

```
  function alterarDigito(texto: string, indice: number) {
    const digito = texto.replace(/\D/g, "").slice(-1);
    setCodigo((atual) =>
      atual.map((item, i) => (i === indice ? digito : item)),
    );
    if (digito && indice < 3) camposCodigo.current[indice + 1]?.focus();
  }

  function voltarDigito(indice: number, tecla: string) {
    if (tecla === "Backspace" && !codigo[indice] && indice > 0) {
      camposCodigo.current[indice - 1]?.focus();
    }
  }
```

Dentro de redefinirSenhaStyle.form, mantenha o bloco dos quatro campos antes das senhas:

```
                <Text style={redefinirSenhaStyle.subtitulo}>
                  Código de verificação
                </Text>
                <View style={redefinirSenhaStyle.linhaCodigo}>
                  {codigo.map((digito, indice) => (
                    <TextInput
                      key={indice}
                      ref={(ref) => {
                        camposCodigo.current[indice] = ref;
                      }}
                      style={redefinirSenhaStyle.campoCodigo}
                      value={digito}
                      onChangeText={(texto) => alterarDigito(texto, indice)}
                      onKeyPress={({ nativeEvent }) =>
                        voltarDigito(indice, nativeEvent.key)
                      }
                      keyboardType="number-pad"
                      maxLength={1}
                      selectTextOnFocus
                      textAlign="center"
                    />
                  ))}
                </View>
```

Etapa 8 — Vincular as senhas aos estados

No campo Nova senha, mantenha value={novaSenha} e onChangeText={setNovaSenha}. No campo Confirmar a senha, mantenha value={confirmarSenha} e onChangeText={setConfirmarSenha}.

```
// Campo de nova senha
value={novaSenha}
onChangeText={setNovaSenha}

// Campo de confirmação
value={confirmarSenha}
onChangeText={setConfirmarSenha}
```

Etapa 9 — Validar o código e redefinir a senha com uma única função

```
  async function salvarNovaSenha() {
    const codigoCompleto = codigo.join("");
    if (!email || typeof email !== "string") {
      setMensagem("E-mail não informado. Solicite um novo código.");
      return;
    }
    if (!/^\d{4}$/.test(codigoCompleto)) {
      setMensagem("Informe os 4 dígitos do código.");
      return;
    }
    if (!novaSenha || !confirmarSenha) {
      setMensagem("Preencha os campos de senha.");
      return;
    }
    if (novaSenha.length < 8) {
      setMensagem("A senha deve ter pelo menos 8 caracteres.");
      return;
    }
```

```
    if (novaSenha !== confirmarSenha) {
      setMensagem("As senhas não correspondem.");
      return;
    }

    setCarregando(true);
    setMensagem("");
    try {
      const respostaCodigo = await fetch(`${API}/auth/validar-codigo`, {
        method: "POST",
        headers: {
          "Content-Type": "application/json",
          Accept: "application/json",
        },
        body: JSON.stringify({ email: email.trim(), codigo: codigoCompleto }),
      });
      const jsonCodigo = await respostaCodigo.json();
      if (!respostaCodigo.ok || !jsonCodigo.success) {
        setMensagem(jsonCodigo.message || "Código inválido ou expirado.");
        return;
      }
      const resetToken = jsonCodigo.data?.reset_token;
      if (!resetToken) {
        setMensagem("Token de redefinição não recebido.");
        return;
      }

      const respostaSenha = await fetch(`${API}/auth/redefinir-senha`, {
        method: "POST",
        headers: {
          "Content-Type": "application/json",
          Accept: "application/json",
        },
        body: JSON.stringify({
          reset_token: resetToken,
          nova_senha: novaSenha,
          nova_senha_confirmation: confirmarSenha,
        }),
      });
      const jsonSenha = await respostaSenha.json();
      if (!respostaSenha.ok || !jsonSenha.success) {
        setMensagem(jsonSenha.message || "Não foi possível redefinir a senha.");
        return;
      }
      router.replace("/");
    } catch (erro) {
      console.log("Erro ao redefinir senha:", erro);
      setMensagem("Não foi possível conectar ao servidor.");
    } finally {
      setCarregando(false);
    }
  }
```

O reset_token é recebido e usado localmente dentro da função. Ele não é o token de login e não deve ser gravado no AsyncStorage.

Etapa 10 — Configurar mensagem e botão Salvar senha

O botão chama salvarNovaSenha() e fica desabilitado enquanto carregando=true. O redirecionamento router.replace("/") acontece somente após sucesso da segunda API.

```
                {mensagem ? (
                  <Text style={redefinirSenhaStyle.erroSenha}>{mensagem}</Text>
                ) : null}

                <Pressable
                  style={({ pressed }) => [
                    redefinirSenhaStyle.btnSalvar,
                    pressed && redefinirSenhaStyle.btnSalvarPressed,
                  ]}
                  onPress={salvarNovaSenha}
                  disabled={carregando}
                >
                  <Text style={redefinirSenhaStyle.textoSalvar}>
                    {carregando ? "Salvando..." : "Salvar senha"}
                  </Text>
                </Pressable>
```

Etapa 11 — Conferir estilos existentes

Os estilos usados são esqueciSenhaStyle.erroSolicitacao, redefinirSenhaStyle.linhaCodigo, campoCodigo e erroSenha. Como os arquivos de estilos não foram anexados, confira se essas propriedades existem nos respectivos arquivos em src/styles/. Não substitua os demais estilos do projeto.